

UNIVERSITY OF OKARA

Department of Computer Science

DATA STRUCTURES & ALGORITHMS

Lab Manual

LAB 07 | Efficient Sorting: Merge Sort, Quick Sort and Counting Sort

Week 7 of 16 • Duration: 3 hours (in-lab) + home practice

Prepared by

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

Applicable to: BSCS • BSIT • BSSE • BSAI • BSDS • BS Cyber Security and all programs offering DSA

Fall 2026 Edition

About This Lab Manual

This manual is part of a **16-week Data Structures & Algorithms (DSA) lab series**. It is designed so that a student starting with **zero programming knowledge** in Week 1 can confidently design, implement and analyse data structures by Week 16. Every lab builds on the previous one, so please do not skip labs.

Language policy: concepts are explained in a language-independent way (diagrams, pseudocode, dry runs). Full implementations are in **Python**, and every lab includes a **C++ Corner** so instructors teaching in C++ can use the same manual.

Every lab follows the same structure

Section	What you get
 Objectives & CLOs	What you will be able to do after this lab
 Motivation	Where this topic is used in real life, industry, research and the future (2026–2030)
 Concepts	Simple explanations with real examples, pseudocode and dry runs
 Code	Python implementation + C++ Corner
 Lab Tasks	Graded tasks:  Easy,  Medium,  Hard
 LeetCode	Real interview problems to practise
 Home Task &  Viva	Practice at home and prepare for oral questions
 Notebooks	Instructor notebook (complete code) and Student notebook (with TODO blanks and auto-check tests)

Lab 07 at a glance

Item	Details
Lab title	Efficient Sorting: Merge Sort, Quick Sort and Counting Sort
Week	7 of 16
Prerequisites	Lab 05 (Elementary Sorting), Lab 06 (Recursion & Divide and Conquer)
Tools	Google Colab / Jupyter / VS Code (Python 3); any C++ compiler for the C++ Corner
Files	Lab07_Instructor.ipynb , Lab07_Student.ipynb
Next lab	Lab 08: Singly Linked List

1. Learning Objectives

By the end of this lab, you will be able to:

1. Implement **Merge Sort** using divide & conquer and the merge procedure.
2. Explain why merge sort is **$O(n \log n)$** using a recursion tree.
3. Implement **Quick Sort** with the **Lomuto partition** and explain pivot choice.
4. Explain quick sort's **average $O(n \log n)$** and **worst $O(n^2)$** cases and how randomisation helps.
5. Implement **Counting Sort**, a non-comparison sort that runs in **$O(n + k)$** .
6. Use **Quickselect** to find the k-th smallest element in average $O(n)$.
7. Compare all sorting algorithms and choose the right one for a situation.

Course Learning Outcomes (CLO) mapping

CLO	Description	Taxonomy level
CLO-2	Implement efficient sorting algorithms	Apply (C3)
CLO-3	Analyse divide-and-conquer sorting algorithms	Analyse (C4)



2. Motivation: From Hours to Seconds

In Lab 05, sorting 2,000 numbers with bubble sort already took noticeable time. Imagine sorting the **crores of CNIC records** at NADRA, or all transactions of a bank in a day. $O(n^2)$ algorithms would take **days**. The algorithms in this lab, which are $O(n \log n)$, do it in **seconds**.

n	n^2 (bubble/insertion)	$n \log_2 n$ (merge/quick)	Speed-up
1,000	1,000,000	$\approx 10,000$	100×
1,000,000	10^{12}	$\approx 2 \times 10^7$	50,000×
100,000,000 (10 crore)	10^{16}	$\approx 2.7 \times 10^9$	≈ 3.7 million×



2.1 Efficient Sorting in Daily Life

- **Sorting exam papers:** teachers often split the pile among helpers, each sorts a small pile, then they **merge** piles. That is merge sort!
- **Organising a library:** pick a letter (pivot), put earlier books on the left and later books on the right, then repeat for each side. That is quick sort.

- **Counting votes:** count how many votes each candidate got, then list them. That is counting sort.



2.2 In Industry

- **Python's `sorted()` (Timsort)** is a hybrid of merge sort and insertion sort. Java uses Timsort for objects and a dual-pivot quick sort for primitive types.
- **C++ `std::sort` (Introsort)** starts with quick sort, switches to heap sort if recursion gets too deep, and uses insertion sort for small pieces.
- **External sorting:** when data is bigger than RAM (e.g., terabytes of logs), databases use merge sort on disk chunks.
- **Big data frameworks** (Hadoop MapReduce, Spark) sort data across many machines using merge-based ideas.



2.3 In Research

- **Parallel and GPU sorting** (e.g., radix sort on GPUs) is an active research area for AI and scientific computing.
- **Learned sorting:** recent research uses machine-learning models to predict where each element belongs, beating classic algorithms on some data.
- Sorting is a building block in **computational geometry** (convex hull) and **GIS** (spatial indexes).



2.4 Why It Matters in 2026–2030



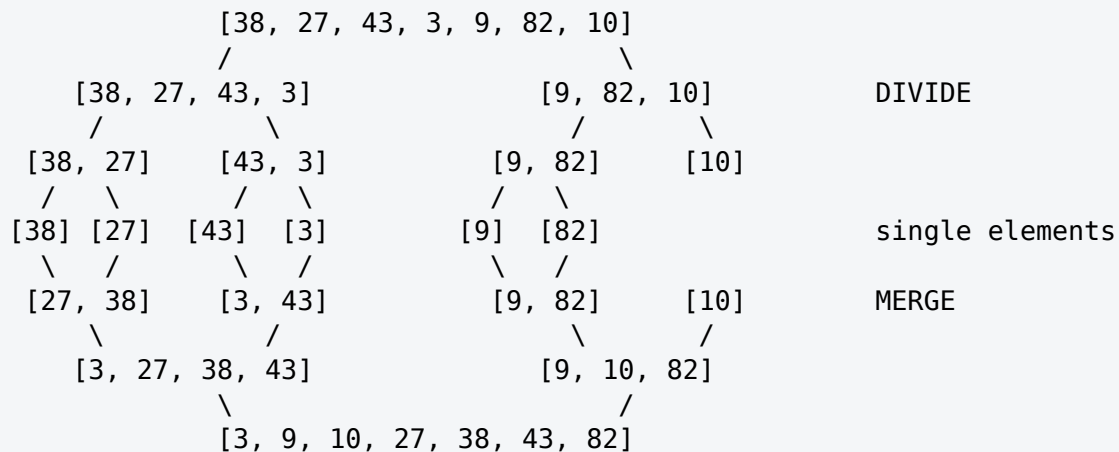
Divide & conquer scales to the cloud

Merge sort's idea, **split the work, solve pieces independently, then combine**, is exactly how modern distributed systems process huge datasets across thousands of computers. Learning it now prepares you for parallel programming, MapReduce and GPU computing.



3. Part A: Merge Sort

Idea: an array of size 1 is already sorted. So: **divide** the array into two halves, **sort each half recursively**, then **merge** the two sorted halves into one sorted array.

Merge sort on [38, 27, 43, 3, 9, 82, 10]

3.1 The Merge Step

You already wrote `merge_sorted` in Lab 03: compare the fronts of two sorted lists and repeatedly take the smaller one. It takes **$O(n)$** time for a total of n elements.

```
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:          # <= keeps merge sort STABLE
            result.append(left[i]); i += 1
        else:
            result.append(right[j]); j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result

def merge_sort(arr):
    if len(arr) <= 1:                    # base case
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])         # sort left half
    right = merge_sort(arr[mid:])        # sort right half
    return merge(left, right)           # combine

print(merge_sort([38, 27, 43, 3, 9, 82, 10]))
```

3.2 Why $O(n \log n)$?

Look at the recursion tree. At **every level**, all n elements are merged once, which costs **$O(n)$** per level. The array is halved at each level, so there are **$\log_2 n$ levels**. Total: **$n \times \log n$** . This holds in the best, average and worst case.

Level	Sub-arrays	Size of each	Merge work at this level
0	1	n	n
1	2	n/2	n
2	4	n/4	n
...	...	...	...
log n	n	1	n

Property	Merge Sort
Time (all cases)	$O(n \log n)$
Extra space	$O(n)$ for merging
Stable?	Yes
In-place?	No
Great for	Linked lists, external (disk) sorting, when stability is needed

⚡ 4. Part B: Quick Sort

Idea: choose a **pivot** element. **Partition** the array so that everything smaller than the pivot is on its left and everything larger is on its right. Now the pivot is in its **final position**. Recursively sort the left and right parts. No merge step is needed.

4.1 Lomuto Partition

Pseudocode

```

ALGORITHM Partition(A, low, high)           // pivot = A[high]
    pivot ← A[high] ; i ← low - 1          // i = end of the "smaller"
    region
    FOR j ← low TO high - 1 DO
        IF A[j] ≤ pivot THEN
            i ← i + 1 ; swap A[i], A[j]
    swap A[i + 1], A[high]                  // put pivot in its final place
    RETURN i + 1

```

Dry run: partition [10, 80, 30, 90, 40, 50, 70] with pivot 70

j	A[j]	A[j] ≤ 70?	i	Array
0	10	Yes → swap A[0], A[0]	0	[10, 80, 30, 90, 40, 50, 70]
1	80	No	0	[10, 80, 30, 90, 40, 50, 70]
2	30	Yes → swap A[1], A[2]	1	[10, 30, 80, 90, 40, 50, 70]
3	90	No	1	[10, 30, 80, 90, 40, 50, 70]
4	40	Yes → swap A[2], A[4]	2	[10, 30, 40, 90, 80, 50, 70]
5	50	Yes → swap A[3], A[5]	3	[10, 30, 40, 50, 80, 90, 70]
end	-	swap A[4], pivot	-	[10, 30, 40, 50, 70 , 90, 80]

```

def partition(a, low, high):
    pivot = a[high]
    i = low - 1
    for j in range(low, high):
        if a[j] <= pivot:
            i += 1
            a[i], a[j] = a[j], a[i]
    a[i + 1], a[high] = a[high], a[i + 1]
    return i + 1

def quick_sort(a, low=0, high=None):
    if high is None:
        high = len(a) - 1
    if low < high:
        p = partition(a, low, high)          # pivot is now at index p
        quick_sort(a, low, p - 1)           # sort left of pivot
        quick_sort(a, p + 1, high)          # sort right of pivot
    return a

print(quick_sort([10, 80, 30, 90, 40, 50, 70]))

```

4.2 Best, Average and Worst Case

Case	When	Recursion depth	Time
Best / average	Pivot splits the array roughly in half	$\log n$	$O(n \log n)$
Worst	Pivot is always the smallest or largest (e.g., already sorted input with last-element pivot)	n	$O(n^2)$

Fix: choose a **random pivot** (or the median of three). Then the bad case becomes extremely unlikely for any input.

```

import random, sys

def quick_sort_random(a, low=0, high=None):
    if high is None:
        high = len(a) - 1
    if low < high:
        r = random.randint(low, high)      # random pivot
        a[r], a[high] = a[high], a[r]
        p = partition(a, low, high)
        quick_sort_random(a, low, p - 1)
        quick_sort_random(a, p + 1, high)
    return a

sys.setrecursionlimit(10_000)
sorted_input = list(range(2000))
print(quick_sort_random(sorted_input[:])[ :10], "... works fast even on sorted input")

```

Property	Quick Sort
Time	$O(n \log n)$ average, $O(n^2)$ worst
Extra space	$O(\log n)$ recursion stack (average)
Stable?	No
In-place?	Yes
Great for	General-purpose in-memory sorting; very fast in practice (cache-friendly)

5. Part C: Counting Sort (Non-Comparison)

Any algorithm that sorts only by **comparing** elements needs at least about $n \log n$ comparisons in the worst case. But if the values are **small integers in a known range** (e.g., marks 0–100, ages 0–120), we can avoid comparisons: just **count** how many times each value occurs.

```
def counting_sort(arr, max_value):
    count = [0] * (max_value + 1)
    for x in arr:
        count[x] += 1
    result = []
    for value in range(max_value + 1):
        result.extend([value] * count[value])
    return result

quiz_marks = [7, 3, 10, 3, 8, 7, 7, 0, 10, 5]
print(counting_sort(quiz_marks, 10))
```

count occurrences: $O(n)$
rebuild: $O(k)$

Time $O(n + k)$ where k is the range of values. Space $O(k)$. It beats $O(n \log n)$ when k is not much larger than n . Counting sort is the building block of **Radix Sort**, which sorts large numbers digit by digit.

6. Part D: Quickselect: The k-th Smallest Without Full Sorting

To find the median or the 3rd highest mark, sorting costs $O(n \log n)$. **Quickselect** uses partition but recurses into **only one side**, giving $O(n)$ on average.

```
def quickselect(a, k):
    # k-th smallest, k starts at 1
    a = a[:]
    low, high = 0, len(a) - 1
    while True:
        r = random.randint(low, high)
        a[r], a[high] = a[high], a[r]
        p = partition(a, low, high)
        if p == k - 1:
            return a[p]
        elif p < k - 1:
            low = p + 1
            # answer is on the right
        else:
            high = p - 1
            # answer is on the left

marks = [67, 89, 45, 92, 78, 55, 81]
print("median:", quickselect(marks, 4), "| 2nd smallest:",
      quickselect(marks, 2))
```



7. Part E: The Complete Sorting Comparison

Algorithm	Best	Average	Worst	Space	Stable	Notes
Bubble	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Teaching only
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No	Fewest swaps
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Small / nearly sorted
Merge	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	Guaranteed, linked lists
Quick	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No	Fastest in practice
Heap (Lab 15)	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No	Guaranteed + in-place
Counting	$O(n + k)$	$O(n + k)$	$O(n + k)$	$O(k)$	Yes*	Small integer range
Timsort (Python)	$O(n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	Real-world default

* Stable when implemented with prefix sums (see the Home Task).

Race on 50,000 random integers between 0 and 100,000

```
import time
data = [random.randint(0, 100_000) for _ in range(50_000)]
for name, f in [("merge_sort", lambda d: merge_sort(d)),
                ("quick_sort_random", lambda d: quick_sort_random(d[:])),
                ("counting_sort", lambda d: counting_sort(d, 100_000)),
                ("sorted (Timsort)", lambda d: sorted(d))]:
    t = time.perf_counter(); out = f(data); e1 = time.perf_counter() - t
    assert out == sorted(data)
    print(f"{name:<18} {e1:.4f} s")
```

8. C++ Corner

C++

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

void mergeSort(vector<int>& a, int l, int r) {
    if (l >= r) return;
    int m = l + (r - l) / 2;
    mergeSort(a, l, m);
    mergeSort(a, m + 1, r);
    vector<int> tmp;
    int i = l, j = m + 1;
    while (i <= m && j <= r) tmp.push_back(a[i] <= a[j] ? a[i++] : a[j++]);
    while (i <= m) tmp.push_back(a[i++]);
    while (j <= r) tmp.push_back(a[j++]);
    for (int k = 0; k < (int)tmp.size(); k++) a[l + k] = tmp[k];
}

int partition(vector<int>& a, int low, int high) {
    int pivot = a[high], i = low - 1;
    for (int j = low; j < high; j++)
        if (a[j] <= pivot) swap(a[++i], a[j]);
    swap(a[i + 1], a[high]);
    return i + 1;
}

void quickSort(vector<int>& a, int low, int high) {
    if (low < high) {
        int p = partition(a, low, high);
        quickSort(a, low, p - 1);
        quickSort(a, p + 1, high);
    }
}

int main() {
    vector<int> a = {38, 27, 43, 3, 9, 82, 10}, b = a;
    mergeSort(a, 0, a.size() - 1);
    quickSort(b, 0, b.size() - 1);
    sort(b.begin(), b.end()); // the library version (Introsort)
    for (int x : a) cout << x << " ";
}
```

9. Common Mistakes

Mistake	Effect	Fix
Using <code><</code> instead of <code><=</code> in merge	Merge sort becomes unstable	Take from the left on ties
Forgetting to copy leftovers after merging	Missing elements	<code>extend(left[i:])</code> and <code>extend(right[j:])</code>
Quick sort with fixed last pivot on sorted data	$O(n^2)$ and RecursionError	Random or median-of-three pivot
Off-by-one in partition range	Infinite recursion	Recurse on <code>p - 1</code> and <code>p + 1</code> , never <code>p</code>
Lomuto partition on data with many duplicates	Degrades towards $O(n^2)$	Use three-way partition (<code><</code> , <code>=</code> , <code>></code> pivot)
Counting sort on huge ranges (e.g., CNIC numbers)	Huge memory	Use radix sort or comparison sort



10. Lab Tasks

Solve all tasks in `Lab07_Student.ipynb`. Each task has a **test cell**: when it prints  your solution is correct. For each task, **write pseudocode first**, then code, then test with your own inputs including edge cases. Difficulty:  Easy  Medium  Hard.

Task 1 Merge Sort with Merge Counting

Write `merge_sort_count(arr)` that returns `(sorted_list, merges)` where `merges` is the number of times the merge step is called. For n elements ($n \geq 1$) it should be exactly $n - 1$. Do not modify the input.

```
merge_sort_count([5, 2, 4, 1]) → ([1, 2, 4, 5], 3)
```

Task 2 Quick Sort in Descending Order

Modify `partition` and `quick_sort` into `partition_desc` and `quick_sort_desc` so that the array is sorted from **largest to smallest**, in place.

```
[3, 9, 1, 7] → [9, 7, 3, 1]
```

Task 3 Stable Counting Sort for Student Records

Sort `(name, marks)` records by marks (0–100) using counting sort with **prefix sums** so that it is **stable**.

Steps: count each mark; convert counts to prefix sums (positions); walk the input **from right to left**, placing each record at `count[mark] - 1` and decrementing.

```
[("Ali",80),("Sara",70),("Hamza",80),("Zoya",70)] → Sara, Zoya, Ali, Hamza
```

Task 4 Merge Overlapping Time Slots

The lab booking system has time slots as `[start, end]` (in minutes). Merge all overlapping slots and return them sorted by start time.

Hint: sort by start time ($O(n \log n)$), then scan once and merge.

```
[[1, 3], [8, 10], [2, 6], [15, 18]] → [[1, 6], [8, 10], [15, 18]]
```

Task 5 🟡 **k-th Highest Marks with Quickselect**

Write `kth_highest(marks, k)` that returns the k-th **highest** value in average $O(n)$ using the quickselect idea (do not sort the whole list). Use it to find the 3rd position holder's marks.

```
kth_highest([67, 89, 45, 92, 78], 2) → 89
```

Task 6 🔴 **Count Inversions in $O(n \log n)$**

In Lab 05 you counted inversions in $O(n^2)$. Do it in **$O(n \log n)$** by modifying merge sort: while merging, whenever you take an element from the **right** half, it forms an inversion with **every remaining element of the left half**.

Return `(sorted_list, inversions)`. Test on 100,000 elements.

```
[2, 4, 1, 3, 5] → ([1, 2, 3, 4, 5], 3)
```

Task 7 🔴 **Largest Number from Digits**

Given non-negative integers, arrange them to form the **largest possible number** and return it as a string.

Hint: a custom comparison: put x before y if `str(x) + str(y) > str(y) + str(x)`. Use `functools.cmp_to_key` with `sorted`, or adapt your merge sort to use this comparison.

```
[3, 30, 34, 5, 9] → "9534330"
```

**11. LeetCode Practice**

Solve these on **leetcode.com** and upload screenshots of the “Accepted” result to your GitHub `DSA-Labs/Lab07` folder.

#	Problem	Level	Concept practised
912	Sort an Array	Medium	Merge sort / quick sort
56	Merge Intervals	Medium	Sort + scan (Task 4)
215	Kth Largest Element in an Array	Medium	Quickselect (Task 5)
179	Largest Number	Medium	Custom comparator (Task 7)
1636	Sort Array by Increasing Frequency	Easy	Sorting with keys

#	Problem	Level	Concept practised
148	Sort List	Medium	Merge sort on a linked list (after Lab 08)



12. Home Task

1. Finish all tasks and push `Lab07_Student.ipynb` to GitHub.
2. Implement **bottom-up (iterative) merge sort** that merges sub-arrays of size 1, 2, 4, 8 ... without recursion.
3. Implement **Hoare's partition** scheme and compare the number of swaps with Lomuto on random data.
4. Implement **Radix Sort** for 6-digit roll numbers using stable counting sort on each digit.
5. **Mini-project: Sorting Visualiser.** Using matplotlib, draw the array as bars after every merge (merge sort) or every partition (quick sort) and save the frames (or animate them in Colab).



13. Viva Questions

1. Explain the three steps of merge sort.
2. Why is merge sort $O(n \log n)$ in all cases?
3. Why does merge sort need $O(n)$ extra space?
4. What is a pivot? What does partition guarantee?
5. When does quick sort take $O(n^2)$? How do we avoid it?
6. Why is quick sort often faster than merge sort in practice even though its worst case is worse?
7. Which of merge sort and quick sort is stable?
8. How does counting sort beat the $O(n \log n)$ lower bound?
9. What is quickselect and what is its average complexity?
10. What algorithm does Python's `sorted()` use?



14. Assessment Rubric

Criteria	Excellent (4)	Good (3)	Satisfactory (2)	Needs work (0–1)
Correctness	All tasks pass all tests	Most tasks correct	Some tasks correct	Mostly incorrect
Logic & pseudocode	Clear pseudocode and dry run for every task	Pseudocode for most tasks	Partial pseudocode	None
Code quality	Meaningful names, functions, comments	Mostly clean	Hard to read	Copied or unreadable
Complexity & edge cases	States Big-O and handles all edge cases	Some edge cases / Big-O	Only basic cases	Not considered
Viva	Explains every line confidently	Explains most parts	Basic understanding	Cannot explain



Lab Rules

- Bring your own laptop with Colab or Jupyter working.
- Arrive on time; late arrival may result in no attendance for the lab.
- Copying another student's code or pasting AI output without understanding it will be awarded zero. You must be able to explain every line in the viva.



15. References and Further Learning

- T. Peters, "Timsort" description:
github.com/python/cpython/blob/main/Objects/listsort.txt
- Sorting animations: visualgo.net/en/sorting
- T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, **Introduction to Algorithms** (CLRS), 4th ed., MIT Press, 2022.
- M. T. Goodrich, R. Tamassia, M. H. Goldwasser, **Data Structures and Algorithms in Python**, Wiley.
- MIT OpenCourseWare 6.006 Introduction to Algorithms; Harvard CS50x (free online courses).

- Visualise code step by step: **pythontutor.com**; animated data structures: **visualgo.net**.



About the Author

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

GitHub: github.com/RamzanPUCIT

LinkedIn: linkedin.com/in/ramzan-ai

ResearchGate: researchgate.net/profile/Muhammad-Ramzan-105

© 2026 Muhammad Ramzan. Prepared for the Department of Computer Science, University of Okara.